

NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Trà Vinh, ngày tháng năm 2026

Giáo viên hướng dẫn

(Ký tên và ghi rõ họ tên)

NHẬN XÉT CỦA THÀNH VIÊN HỘI ĐỒNG

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Trà Vinh, ngày tháng năm 2026

Thành viên hội đồng

(Ký tên và ghi rõ họ tên)

LỜI CẢM ƠN

Trong suốt quá trình học tập và thực hiện đồ án môn học Chuyên đề ASP.NET, em đã nhận được sự hướng dẫn, hỗ trợ và động viên từ quý thầy cô, gia đình và bạn bè. Những sự giúp đỡ đó là điều kiện quan trọng giúp em hoàn thành đề tài “Ứng dụng quản lý thu chi cá nhân”.

Trước hết, em xin gửi lời cảm ơn chân thành đến thầy TS. Đoàn Phước Miên, giảng viên hướng dẫn đã tận tình định hướng, góp ý và hỗ trợ em trong quá trình phân tích yêu cầu, thiết kế hệ thống, xây dựng chương trình và hoàn thiện báo cáo đồ án.

Em cũng xin cảm ơn quý thầy cô Khoa Công nghệ Thông tin, Trường Kỹ thuật và Công nghệ, Đại học Trà Vinh đã truyền đạt những kiến thức nền tảng về lập trình web, cơ sở dữ liệu, phân tích thiết kế hệ thống và triển khai ứng dụng. Đây là cơ sở quan trọng để em vận dụng vào việc xây dựng sản phẩm phần mềm trong đồ án.

Do thời gian thực hiện có hạn và kiến thức còn hạn chế, báo cáo và chương trình không tránh khỏi thiếu sót. Em rất mong nhận được những ý kiến đóng góp của quý thầy cô để đề tài được hoàn thiện hơn.

Em xin chân thành cảm ơn!

MỤC LỤC

MỞ ĐẦU	1
1. Bối cảnh và lý do chọn đề tài	1
2. Mục tiêu của đề tài	1
3. Đối tượng và phạm vi nghiên cứu	2
4. Các vai trò sử dụng hệ thống	2
Bảng 1.1: Vai trò người dùng của hệ thống	2
5. Ý nghĩa thực tiễn của đề tài	2
CHƯƠNG 1: NGHIÊN CỨU LÝ THUYẾT	4
1.1. Nền tảng ASP.NET Framework 4.8	4
1.2. Ngôn ngữ C# và lập trình hướng đối tượng	4
1.3. ADO.NET và truy cập dữ liệu SQL Server	4
1.4. SQL Server và thiết kế cơ sở dữ liệu quan hệ	5
1.5. IIS và mô hình triển khai ứng dụng web local	5
1.6. MVC-style trong phạm vi đồ án	5
1.7. Frontend HTML, CSS và JavaScript thuần	6
1.8. Session, xác thực và phân quyền	6
CHƯƠNG 2: HIỆN THỰC HÓA NGHIÊN CỨU	8
2.1. Mô tả bài toán	8
2.2. Yêu cầu chức năng	8
Bảng 2.1: Yêu cầu chức năng chính	8
2.3. Yêu cầu phi chức năng	9
Bảng 2.2: Yêu cầu phi chức năng	9
2.4. Sơ đồ Use Case	10

Bảng 2.3: Tác nhân và Use Case chính	10
2.5. Sơ đồ luồng dữ liệu	11
<i>Hình 2.5.1: DFD mức ngữ cảnh của hệ thống</i>	11
<i>Hình 2.5.2: DFD mức 1 của hệ thống</i>	12
2.6. Thiết kế cơ sở dữ liệu	12
<i>Hình 2.6: Sơ đồ ERD cơ sở dữ liệu</i>	13
2.6.1. Bảng Users	13
2.6.2. Bảng TransactionTypes	14
2.6.3. Bảng Categories	14
2.6.4. Bảng Transactions	15
2.6.5. Bảng SavingsGoals	16
2.7. Thiết kế kiến trúc chương trình	16
2.8. Thiết kế giao diện	17
<i>Hình 2.8.1: Màn hình đăng nhập</i>	18
<i>Hình 2.8.2: Màn hình Quản lý giao dịch</i>	18
<i>Hình 2.8.3: màn hình biểu đồ giao dịch</i>	19
<i>Hình 2.8.4: màn hình các giao dịch đã tạo</i>	20
<i>Hình 2.8.5: Màn hình Lập kế hoạch tiết kiệm</i>	20
<i>Hình 2.8.6: Màn hình quản lý danh mục giao dịch</i>	20
<i>Hình 2.8.7 Màn hình quản lý loại thu / chi</i>	21
<i>Hình 2.8.8 Màn hình quản lý user</i>	21
2.9. Thiết kế API	22
2.10. Luồng kiểm tra số dư lũy kế	23
2.11. Kiểm tra dữ liệu nhập	24

2.12 Thiết kế dữ liệu mẫu	25
2.12.1. Bảng Tài khoản và dữ liệu mẫu	Error! Bookmark not defined.
2.13. Cách thức triển khai	25
CHƯƠNG 3: KẾT QUẢ NGHIÊN CỨU	27
3.1. Các chức năng đã hoàn thành	27
3.1.1. Bảng Tổng hợp kết quả chức năng	27
3.2. Kịch bản kiểm thử	28
3.2.1. Bảng kịch bản kiểm thử chức năng	28
3.3. Đánh giá giao diện và trải nghiệm	28
3.4. Đánh giá triển khai	29
3.5. Hạn chế còn tồn tại	29
CHƯƠNG 4: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	30
4.1. Kết luận	30
4.2. Hướng phát triển	30
4.3. Tài liệu tham khảo	31
PHỤ LỤC A: CẤU TRÚC MÃ NGUỒN	32
Bảng A: Mô tả các thư mục trên github	32
PHỤ LỤC B: PHÂN TÍCH LỖI THƯỜNG GẶP KHI TRIỂN KHAI	33
Bảng B: Lỗi triển khai thường gặp và cách xử lý	33

DANH MỤC HÌNH ẢNH

<i>Hình 2.5.1: DFD mức ngữ cảnh của hệ thống</i>	11
<i>Hình 2.5.2: DFD mức 1 của hệ thống</i>	12
<i>Hình 2.6.1: Sơ đồ ERD cơ sở dữ liệu</i>	13
<i>Hình 2.8.1: Màn hình đăng nhập</i>	18
<i>Hình 2.8.2: Màn hình Quản lý giao dịch</i>	18
<i>Hình 2.8.3: màn hình biểu đồ giao dịch</i>	19
<i>Hình 2.8.4: màn hình các giao dịch đã tạo</i>	20
<i>Hình 2.8.5: Màn hình Lập kế hoạch tiết kiệm</i>	20
<i>Hình 2.8.6: Màn hình quản lý danh mục giao dịch</i>	20
<i>Hình 2.8.7 Màn hình quản lý loại thu / chi</i>	21
<i>Hình 2.8.8 Màn hình quản lý user</i>	21

DANH MỤC BẢNG BIỂU

Bảng 1.1: Vai trò người dùng của hệ thống	2
Bảng 2.1: Yêu cầu chức năng chính	8
Bảng 2.2: Yêu cầu phi chức năng	9
Bảng 2.3: Tác nhân và Use Case chính	10
Bảng 2.6.1: Users	13
Bảng 2.6.2: TransactionTypes	14
Bảng 2.6.3: Categories	14
Bảng 2.6.4: Transactions	15
Bảng 2.6.5: SavingsGoals.....	16
Bảng 2.7.1: Các thành phần kiến trúc.....	17
Bảng 2.8.1: Danh sách màn hình chính của hệ thống	22
Bảng 2.9.1: Danh sách API chính	22
Bảng 2.12.1: Tài khoản và dữ liệu mẫu.....	25
Bảng 3.1.1: Tổng hợp kết quả chức năng.....	27
Bảng 3.2.1: kịch bản kiểm thử chức năng	28
Bảng A: Mô tả các thư mục trên github	32
Bảng B: Lỗi triển khai thường gặp và cách xử lý	33

TÓM TẮT ĐỒ ÁN

Đồ án “Ứng dụng quản lý thu chi cá nhân” tập trung xây dựng một website hỗ trợ người dùng ghi nhận và theo dõi tình hình tài chính cá nhân thông qua các khoản thu nhập, chi tiêu và tiết kiệm. Hệ thống được phát triển nhằm thay thế cách ghi chép thủ công, giúp dữ liệu được lưu trữ tập trung, dễ tìm kiếm, dễ thống kê và có khả năng cảnh báo khi người dùng chi tiêu hoặc tiết kiệm vượt quá số dư hiện có.

Ứng dụng được xây dựng bằng ASP.NET Framework 4.8, C#, ADO.NET và SQL Server, triển khai trên IIS trong môi trường Windows 10/11. Phần giao diện sử dụng HTML, CSS và JavaScript thuần, không phụ thuộc vào thư viện trực tuyến, đảm bảo chạy được trong môi trường offline. Source code được tổ chức theo hướng MVC-style, tách rõ phần Model, View và Controller để dễ bảo trì và mở rộng.

Các chức năng chính của hệ thống gồm đăng nhập, phân quyền Admin/User, quản lý giao dịch, quản lý kế hoạch tiết kiệm, quản lý danh mục, quản lý loại thu chi, quản lý người dùng, thống kê theo thời gian, biểu đồ trực quan và kiểm tra số dư lũy kế. Kết quả thực nghiệm cho thấy hệ thống đáp ứng được yêu cầu đề tài, có thể triển khai trên máy cá nhân hoặc mạng nội bộ phục vụ nhu cầu quản lý tài chính cá nhân cơ bản.

MỞ ĐẦU

1. Bối cảnh và lý do chọn đề tài

Trong đời sống hằng ngày, việc theo dõi thu nhập, chi tiêu và khoản tiết kiệm có vai trò quan trọng đối với mỗi cá nhân và gia đình. Tuy nhiên, nhiều người vẫn ghi chép thủ công bằng sổ tay, bảng tính rời rạc hoặc chỉ nhớ bằng kinh nghiệm. Cách làm này dễ thiếu sót dữ liệu, khó tổng hợp theo tháng và khó đánh giá xu hướng chi tiêu trong dài hạn.

Sự phổ biến của máy tính cá nhân và trình duyệt web tạo điều kiện thuận lợi để xây dựng các ứng dụng quản lý tài chính cá nhân chạy cục bộ. Với yêu cầu của học phần Chuyên đề ASP.NET, đề tài “Ứng dụng quản lý thu chi cá nhân” được lựa chọn nhằm vận dụng kiến thức lập trình web ASP.NET, cơ sở dữ liệu SQL Server, thiết kế giao diện và phân tích hệ thống vào một bài toán gần gũi, có tính thực tiễn.

Ứng dụng được định hướng chạy trên Windows 10/11, triển khai bằng IIS và SQL Server Express hoặc SQL Server. Hệ thống không sử dụng dịch vụ AI, không phụ thuộc CDN hoặc thư viện trực tuyến, phù hợp với môi trường học tập, demo hoặc máy cá nhân không có Internet.

2. Mục tiêu của đề tài

Mục tiêu tổng quát của đề tài là xây dựng một website quản lý thu chi cá nhân giúp người dùng ghi nhận giao dịch, phân loại theo danh mục, xem thống kê và lập kế hoạch tiết kiệm. Ứng dụng phải có giao diện dễ sử dụng, dữ liệu được lưu trữ tập trung trong SQL Server và có thể cài đặt lại bằng script tự động.

Các mục tiêu cụ thể gồm: xây dựng cơ sở dữ liệu quản lý người dùng, loại giao dịch, danh mục, giao dịch và kế hoạch tiết kiệm; phát triển chức năng đăng nhập và phân quyền Admin/User; cho phép thêm, sửa, xóa, tìm kiếm và phân trang giao dịch; hiển thị thống kê thu, chi, tiết kiệm; kiểm tra số dư khi người dùng tạo khoản chi hoặc khoản tiết kiệm; hỗ trợ dữ liệu mẫu phục vụ demo.

3. Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu là quy trình quản lý thu nhập, chi tiêu, tiết kiệm của cá nhân trong phạm vi ứng dụng web cục bộ. Hệ thống tập trung vào các nghiệp vụ cốt lõi như ghi nhận giao dịch, phân loại giao dịch, lập kế hoạch tiết kiệm, thống kê và kiểm soát số dư.

Phạm vi đề tài không bao gồm kết nối ngân hàng, thanh toán trực tuyến, đồng bộ đa thiết bị, nhận diện hóa đơn, dự báo bằng AI hoặc quản lý tài sản phức tạp. Các chức năng được giới hạn ở mức phù hợp với một đồ án ASP.NET: đầy đủ về phân tích, thiết kế, cài đặt, kiểm thử và triển khai trên môi trường local.

4. Các vai trò sử dụng hệ thống

Bảng 1.1: Vai trò người dùng của hệ thống

Vai trò	Mô tả	Phạm vi chức năng
Admin	Người quản trị hệ thống	Quản lý giao dịch, kế hoạch tiết kiệm, danh mục, loại thu/chi và người dùng.
User thường	Người sử dụng cá nhân	Quản lý giao dịch và kế hoạch tiết kiệm của chính mình; không được quản lý danh mục, loại thu/chi và user.

5. Ý nghĩa thực tiễn của đề tài

Về mặt thực tiễn, ứng dụng giúp người dùng hình thành thói quen ghi chép tài chính cá nhân, nắm rõ nguồn thu, nhóm chi tiêu và khả năng tiết kiệm. Việc có cảnh báo khi khoản chi hoặc khoản tiết kiệm vượt số dư lũy kế giúp người dùng tránh nhập dữ liệu thiếu hợp lý và hỗ trợ kiểm soát tài chính tốt hơn.

Về mặt học thuật, đề tài giúp sinh viên thực hành quy trình xây dựng phần mềm hoàn chỉnh: phân tích yêu cầu, thiết kế cơ sở dữ liệu, thiết kế giao diện, lập trình

backend/frontend, phân quyền, kiểm thử và viết tài liệu triển khai. Đây là nền tảng quan trọng để tiếp tục phát triển các hệ thống quản lý thực tế sau này.

CHƯƠNG 1: NGHIÊN CỨU LÝ THUYẾT

1.1. Nền tảng ASP.NET Framework 4.8

ASP.NET Framework 4.8 là nền tảng phát triển ứng dụng web của Microsoft chạy trên .NET Framework truyền thống. Nền tảng này phù hợp với môi trường Windows và IIS, hỗ trợ Web Forms, Web Handler, Web API kiểu cổ điển, Session, cấu hình Web.config và khả năng tích hợp với SQL Server.

Trong đề tài, ASP.NET Framework 4.8 được sử dụng để xây dựng backend thông qua các endpoint .ashx. Mỗi endpoint nhận request HTTP từ frontend, kiểm tra session, xử lý nghiệp vụ và trả về dữ liệu JSON. Cách tiếp cận này đơn giản, dễ triển khai offline và không cần thêm package từ Internet.

1.2. Ngôn ngữ C# và lập trình hướng đối tượng

C# là ngôn ngữ lập trình chính trong backend. Các lớp xử lý được viết theo phong cách hướng đối tượng, tách các phần xử lý chung như kết nối cơ sở dữ liệu, đọc JSON, trả JSON, kiểm tra đăng nhập và phân quyền vào các lớp dùng lại.

Việc sử dụng C# giúp hệ thống có kiểu dữ liệu rõ ràng, dễ kiểm soát lỗi khi xử lý số tiền, ngày tháng, trạng thái người dùng và vai trò. Các phương thức kiểm tra số dư lũy kế, kiểm tra ngày hợp lệ, kiểm tra quyền admin được đặt ở backend nhằm đảm bảo tính an toàn ngay cả khi người dùng bỏ qua validate phía trình duyệt.

1.3. ADO.NET và truy cập dữ liệu SQL Server

ADO.NET là thư viện truy cập dữ liệu truyền thống của .NET Framework. Trong ứng dụng, ADO.NET được sử dụng để thực thi câu lệnh SQL, lấy DataTable, lấy giá trị scalar và truyền tham số an toàn qua SqlParameter. Cách làm này phù hợp với yêu cầu không sử dụng thư viện ngoài và giúp sinh viên hiểu rõ cách ứng dụng trao đổi trực tiếp với cơ sở dữ liệu.

Các truy vấn trong hệ thống được viết bằng T-SQL. Những thao tác thêm, sửa, xóa sử dụng câu lệnh INSERT, UPDATE, DELETE; các màn hình danh sách sử dụng

SELECT kết hợp JOIN giữa Transactions, Users, TransactionTypes và Categories. Phân trang giao dịch sử dụng OFFSET/FETCH của SQL Server.

1.4. SQL Server và thiết kế cơ sở dữ liệu quan hệ

SQL Server là hệ quản trị cơ sở dữ liệu quan hệ do Microsoft phát triển. Phiên bản SQL Server Express có thể sử dụng miễn phí cho môi trường học tập và ứng dụng nhỏ. Trong đề tài, SQL Server lưu trữ toàn bộ dữ liệu người dùng, danh mục, loại giao dịch, giao dịch và kế hoạch tiết kiệm.

Thiết kế cơ sở dữ liệu dựa trên các nguyên tắc quan hệ: mỗi bảng có khóa chính, các bảng nghiệp vụ liên kết bằng khóa ngoại, những trường bắt buộc được ràng buộc NOT NULL, và một số trường như Username, TypeCode có tính duy nhất. Việc tổ chức dữ liệu như vậy giúp giảm trùng lặp, đảm bảo tính toàn vẹn và thuận lợi cho thống kê.

1.5. IIS và mô hình triển khai ứng dụng web local

Internet Information Services (IIS) là web server của Microsoft trên Windows. IIS cho phép tạo website, application pool, virtual directory và cấu hình backend ASP.NET. Trong đề tài, source được triển khai tại C:\inetpub\wwwroot\PersonalFinanceOffline, trong đó frontend là thư mục tĩnh, backend được convert thành Application để ASP.NET xử lý .ashx.

Application Pool chạy dưới tài khoản dạng IIS APPPOOL\TênPool. Vì vậy, để backend truy cập SQL Server bằng Integrated Security, cần cấp quyền login và quyền database cho tài khoản App Pool tương ứng. Đây là một nội dung quan trọng trong quá trình triển khai ứng dụng ASP.NET trên IIS.

1.6. MVC-style trong phạm vi đề án

Kiến trúc MVC-style của hệ thống được chia thành ba nhóm chính:

View:

Tầng View nằm trong thư mục frontend, gồm file index.html và các file trong frontend/assets như app.js, style.css. Đây là phần giao diện người dùng, cho

phép người dùng đăng nhập, nhập giao dịch, lọc dữ liệu, xem thống kê, quản lý kế hoạch tiết kiệm và thực hiện các thao tác theo quyền được cấp.

Controller:

Tầng Controller được triển khai thông qua các file .ashx trong thư mục backend/api và các lớp C# xử lý tương ứng trong backend/App_Code. Các endpoint như login.ashx, transactions.ashx, savingsGoals.ashx, categories.ashx, transactionTypes.ashx, users.ashx tiếp nhận request từ frontend, sau đó gọi các lớp xử lý nghiệp vụ để thực hiện thao tác tương ứng.

Model/Data:

Tầng Model/Data bao gồm cơ sở dữ liệu SQL Server và các đoạn xử lý truy vấn thông qua ADO.NET. Các bảng chính gồm Users, TransactionTypes, Categories, Transactions, SavingsGoals. Dữ liệu được truy xuất thông qua lớp hỗ trợ kết nối trong Db.cs, sau đó được trả về dưới dạng JSON để frontend hiển thị.

1.7. Frontend HTML, CSS và JavaScript thuần

Frontend của ứng dụng sử dụng HTML5, CSS3 và JavaScript thuần. Toàn bộ tài nguyên được lưu local trong thư mục frontend/assets, không gọi CDN và không tải thư viện từ Internet. Điều này giúp website vẫn hoạt động trong môi trường máy tính không có kết nối mạng.

JavaScript chịu trách nhiệm gọi API .ashx bằng fetch, nhận dữ liệu JSON, render bảng giao dịch, biểu đồ canvas, form nhập liệu, thông báo lỗi và phân trang. CSS định nghĩa bố cục sidebar, card, bảng dữ liệu, màu sắc thu/chi/tiết kiệm, badge phân quyền và các thành phần cảnh báo.

1.8. Session, xác thực và phân quyền

Sau khi đăng nhập thành công, backend lưu UserId, Username và IsAdmin vào Session. Mỗi API nghiệp vụ đều kiểm tra session bằng hàm RequireLogin. Các API dành riêng cho quản trị viên như quản lý user, thêm/sửa/xóa danh mục và loại thu chi sử dụng thêm RequireAdmin.

Frontend cũng nhận thông tin isAdmin để ẩn menu không phù hợp. User thường chỉ thấy menu Giao dịch và Kế hoạch tiết kiệm; Admin thấy thêm Danh mục, Loại thu/chi và User. Việc kiểm tra cả frontend và backend giúp tăng tính an toàn: frontend tạo trải nghiệm phù hợp, backend đảm bảo người dùng không thể gọi API trái quyền.

CHƯƠNG 2: HIỆN THỰC HÓA NGHIÊN CỨU

2.1. Mô tả bài toán

Bài toán đặt ra là xây dựng một website giúp cá nhân quản lý thu nhập, chi tiêu và tiết kiệm. Người dùng cần nhập các khoản phát sinh hằng ngày, phân loại theo loại giao dịch và danh mục, sau đó xem lại dữ liệu theo thời gian, theo danh mục hoặc theo từ khóa ghi chú.

Ngoài quản lý giao dịch, hệ thống cần hỗ trợ lập kế hoạch tiết kiệm. Người dùng có thể đặt mục tiêu, số tiền cần đạt, ngân sách chi tiêu tháng, ngày bắt đầu và ngày đạt mục tiêu. Hệ thống theo dõi tiến độ dựa trên các giao dịch loại Tiết kiệm và đưa ra cảnh báo nếu chi tiêu tháng hiện tại vượt ngân sách.

Hệ thống cũng cần có cơ chế phân quyền. Admin có quyền quản lý toàn bộ dữ liệu cấu hình như user, danh mục và loại thu chi. User thường chỉ quản lý dữ liệu cá nhân của mình. Mỗi user chỉ nhìn thấy giao dịch và kế hoạch tiết kiệm thuộc tài khoản đang đăng nhập.

2.2. Yêu cầu chức năng

Bảng 2.1: Yêu cầu chức năng chính

Mã	Chức năng	Mô tả
F01	Đăng nhập/đăng xuất	Xác thực tài khoản, tạo session và phân quyền theo IsAdmin.
F02	Quản lý giao dịch	Thêm, sửa, xóa, lọc, tìm kiếm và phân trang giao dịch.
F03	Quản lý kế hoạch tiết kiệm	Tạo mục tiêu, theo dõi tiến độ, cảnh báo và gợi ý tiết kiệm.
F04	Thống kê và biểu đồ	Hiển thị tổng thu, tổng chi, tiết kiệm, số dư và biểu đồ theo ngày/tháng/năm.

F05	Quản lý danh mục	Admin thêm, sửa, xóa danh mục theo loại giao dịch.
F06	Quản lý loại thu/chi	Admin quản lý các loại Income, Expense, Saving.
F07	Quản lý user	Admin tạo/sửa/xóa user, đặt trạng thái hoạt động và phân quyền.
F08	Kiểm tra số dư	Chặn khoản chi hoặc tiết kiệm nếu vượt số dư lũy kế đến tháng giao dịch.

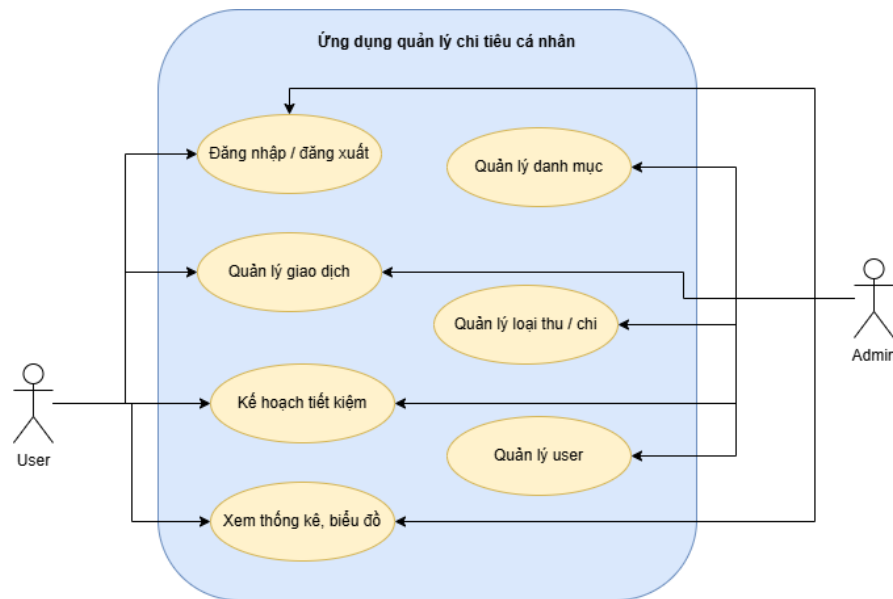
2.3. Yêu cầu phi chức năng

Bảng 2.2: Yêu cầu phi chức năng

Mã	Yêu cầu	Cách đáp ứng
NF01	Chạy offline	Không dùng CDN, toàn bộ CSS/JS local.
NF02	Dễ triển khai	Có script deploy lên IIS và chạy SQL tự động.
NF03	Bảo mật cơ bản	Session login, phân quyền Admin/User, backend kiểm tra quyền.
NF04	Dễ bảo trì	Tổ chức MVC-style gồm Models, Controllers, Core và View.
NF05	Tính đúng dữ liệu	Ràng buộc khóa chính, khóa ngoại, NOT NULL và validate nghiệp vụ.
NF06	Hiệu năng chấp nhận	Phân trang 10/20/50 giao dịch, query có filter và OFFSET/FETCH.

2.4. Sơ đồ Use Case

Use Case tổng quát mô tả hai tác nhân chính: Admin và User thường. Admin có đầy đủ chức năng quản lý dữ liệu hệ thống, trong khi User thường chỉ thực hiện các nghiệp vụ cá nhân gồm giao dịch và kế hoạch tiết kiệm. Việc tách vai trò giúp hệ thống kiểm soát quyền truy cập và tránh người dùng thường thay đổi cấu hình chung.



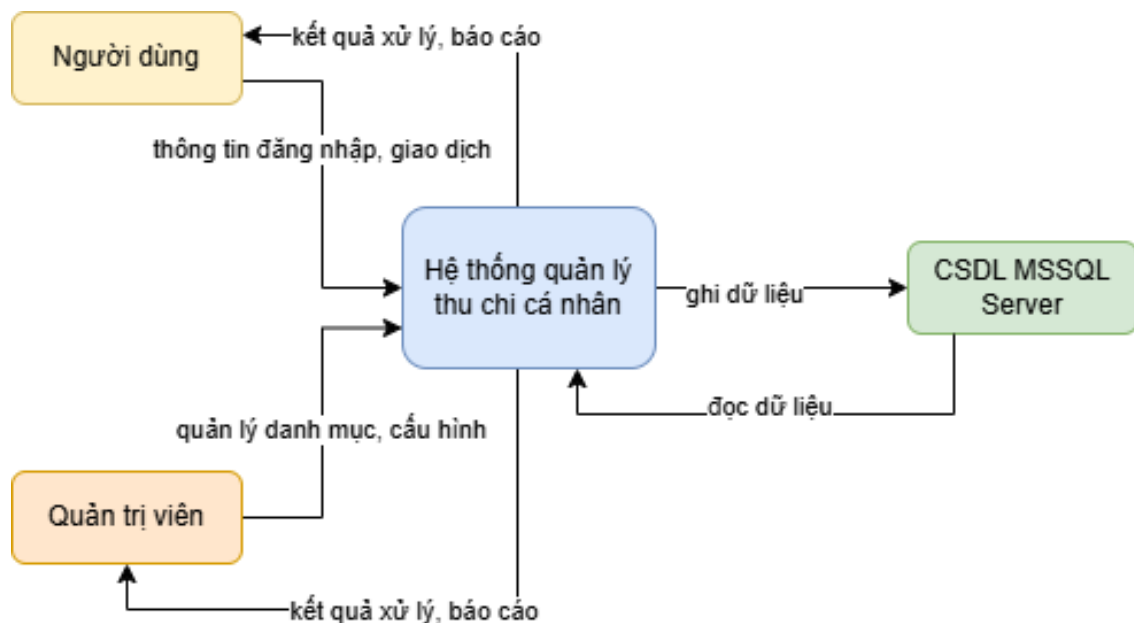
Hình 2.4.1: Sơ đồ Use Case tổng quát của hệ thống

Bảng 2.3: Tác nhân và Use Case chính

Tác nhân	Use Case chính
Admin	Đăng nhập, quản lý giao dịch, kế hoạch tiết kiệm, danh mục, loại thu/chi, user, import dữ liệu mẫu.
User thường	Đăng nhập, thêm/sửa/xóa giao dịch cá nhân, lập kế hoạch tiết kiệm và xem thống kê cá nhân.

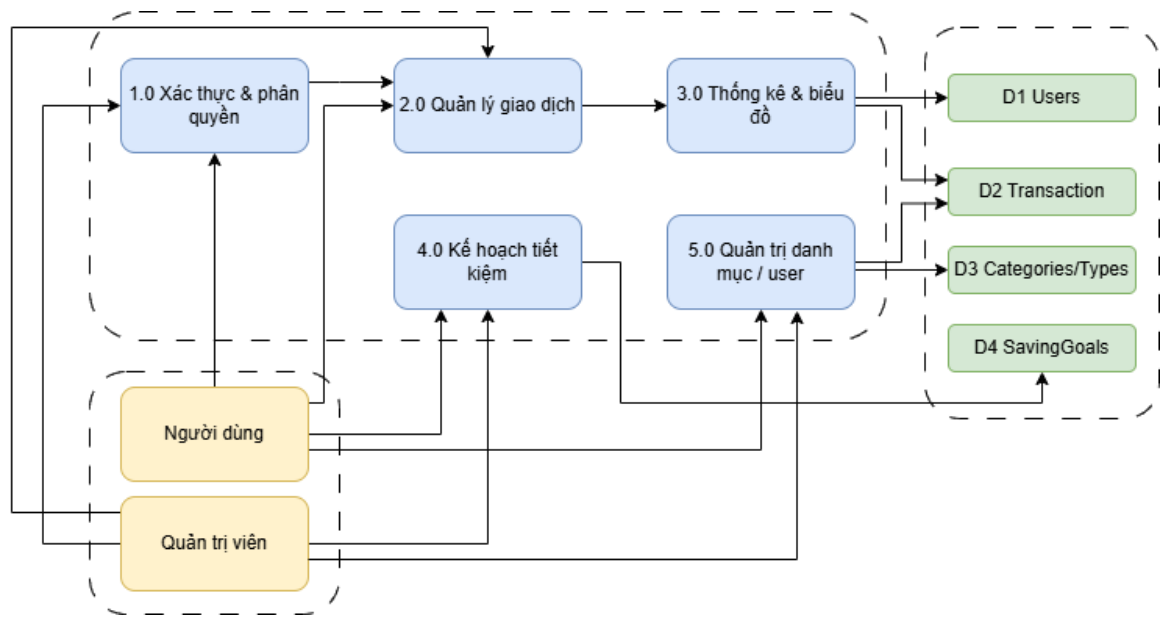
2.5. Sơ đồ luồng dữ liệu

Sơ đồ DFD mức ngữ cảnh cho thấy hệ thống là trung tâm xử lý giữa người dùng, quản trị viên và cơ sở dữ liệu SQL Server. Người dùng gửi thông tin đăng nhập, giao dịch và kế hoạch; hệ thống phản hồi kết quả xử lý, thống kê và cảnh báo. Admin gửi dữ liệu quản trị như danh mục, loại giao dịch và user. Admin gửi dữ liệu quản trị như danh mục, loại giao dịch và user.



Hình 2.5.1: DFD mức ngữ cảnh của hệ thống

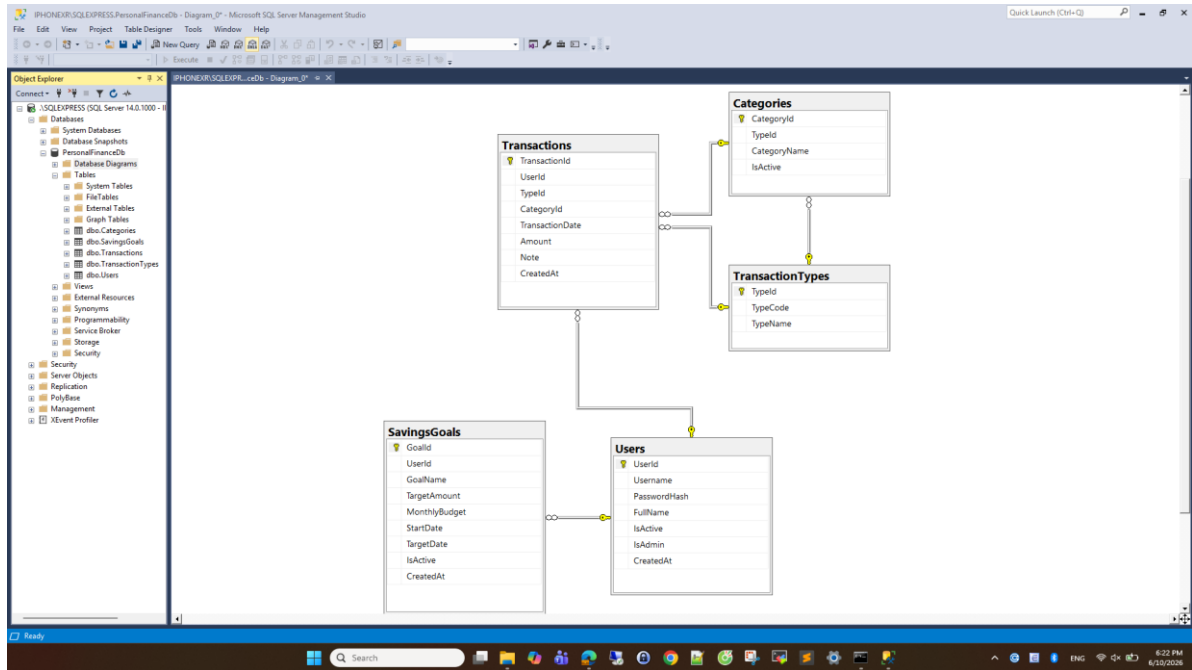
DFD mức 1 phân rã hệ thống thành các tiến trình nhỏ: xác thực và phân quyền, quản lý giao dịch, thống kê và biểu đồ, kế hoạch tiết kiệm, quản trị danh mục và user. Các tiến trình này đọc/ghi vào các kho dữ liệu Users, Transactions, Categories/Types và SavingsGoals.



Hình 2.5.2: DFD mức 1 của hệ thống

2.6. Thiết kế cơ sở dữ liệu

Cơ sở dữ liệu PersonalFinanceDb gồm năm bảng chính: Users, TransactionTypes, Categories, Transactions và SavingsGoals. Thiết kế này đủ để quản lý tài khoản, phân quyền, loại giao dịch, danh mục, giao dịch tài chính và kế hoạch tiết kiệm. Các bảng được liên kết với nhau bằng khóa ngoại nhằm đảm bảo dữ liệu có quan hệ rõ ràng.



Hình 2.6.1: Sơ đồ ERD cơ sở dữ liệu

Bảng 2.6.1: Users

Cột	Kiểu dữ liệu	Ràng buộc	Mô tả
UserId	INT	PK, IDENTITY	Khóa chính tự tăng của người dùng.
Username	NVARCHAR(50)	NOT NULL, UNIQUE	Tên đăng nhập duy nhất.
PasswordHash	NVARCHAR(64)	NOT NULL	Mật khẩu đã băm SHA-256.
FullName	NVARCHAR(100)	NULL	Họ tên hiển thị.
IsActive	BIT	NOT NULL, DEFAULT 1	Trạng thái hoạt động của tài khoản.
IsAdmin	BIT	NOT NULL,	Xác định tài khoản có

		DEFAULT 0	quyền quản trị hay không.
CreatedAt	DATETIME	DEFAULT GETDATE()	Ngày tạo tài khoản.

Bảng 2.6.2: TransactionTypes

Cột	Kiểu dữ liệu	Ràng buộc	Mô tả
TypeId	INT	PK, IDENTITY	Khóa chính của loại giao dịch.
TypeCode	NVARCHAR(20)	NOT NULL, UNIQUE	Mã loại: Income, Expense, Saving.
TypeName	NVARCHAR(50)	NOT NULL	Tên loại hiển thị: Thu nhập, Chi tiêu, Tiết kiệm.

Bảng 2.6.3: Categories

Cột	Kiểu dữ liệu	Ràng buộc	Mô tả
CategoryId	INT	PK, IDENTITY	Khóa chính của danh mục.
TypeId	INT	FK -> TransactionTypes.TypeId	Danh mục thuộc loại giao dịch nào.
CategoryName	NVARCHAR(100)	NOT NULL	Tên danh mục, ví dụ Lương, Ăn uống, Tiền mặt.

IsActive	BIT	NOT NULL, DEFAULT 1	Trạng thái sử dụng của danh mục.
----------	-----	------------------------	-------------------------------------

Bảng 2.6.4: Transactions

Cột	Kiểu dữ liệu	Ràng buộc	Mô tả
TransactionId	INT	PK, IDENTITY	Khóa chính của giao dịch.
UserId	INT	FK -> Users.UserId	Người dùng sở hữu giao dịch.
TypeId	INT	FK -> TransactionTypes.TypeId	Loại giao dịch.
CategoryId	INT	FK -> Categories.CategoryId	Danh mục giao dịch.
TransactionDate	DATE	NOT NULL	Ngày phát sinh giao dịch.
Amount	DECIMAL(18,2)	NOT NULL	Số tiền giao dịch.
Note	NVARCHAR(500)	NULL	Ghi chú mô tả giao dịch.
CreatedAt	DATETIME	DEFAULT GETDATE()	Thời điểm tạo bản ghi.

Bảng 2.6.5: SavingsGoals

Cột	Kiểu dữ liệu	Ràng buộc	Mô tả
GoalId	INT	PK, IDENTITY	Khóa chính của mục tiêu tiết kiệm.
UserId	INT	FK -> Users.UserId	Người dùng sở hữu mục tiêu.
GoalName	NVARCHAR(150)	NOT NULL	Tên mục tiêu, ví dụ Quỹ dự phòng.
TargetAmount	DECIMAL(18,2)	NOT NULL	Số tiền cần đạt.
MonthlyBudget	DECIMAL(18,2)	NOT NULL	Ngân sách chi tiêu tháng.
StartDate	DATE	NOT NULL	Ngày bắt đầu mục tiêu.
TargetDate	DATE	NOT NULL	Ngày dự kiến đạt mục tiêu.
IsActive	BIT	NOT NULL, DEFAULT 1	Trạng thái đang theo dõi hay tạm dừng.
CreatedAt	DATETIME	DEFAULT GETDATE()	Ngày tạo mục tiêu.

2.7. Thiết kế kiến trúc chương trình

Source code được tổ chức theo mô hình MVC-style. Thư mục backend/api chứa các file .ashx đóng vai trò endpoint. Các endpoint này gọi đến các controller trong backend/App_Code/Controllers. Thư mục Models chứa các lớp mô tả thực thể dữ liệu, Core chứa Db.cs và WebUtil.cs để dùng chung.

Frontend nằm trong thư mục frontend, gồm index.html, assets/style.css và assets/app.js. File index.html định nghĩa giao diện, app.js gọi API backend và render dữ liệu, style.css đảm nhiệm bố cục và màu sắc.

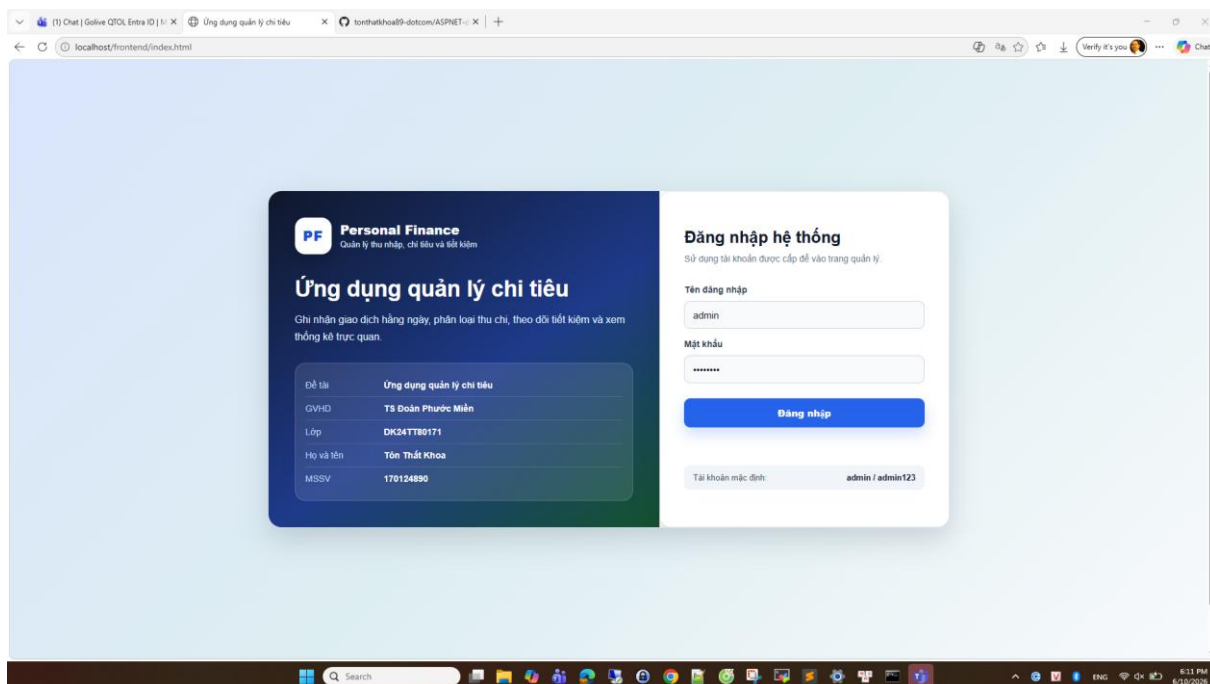
Bảng 2.7.1: Các thành phần kiến trúc

Thành phần	Đường dẫn	Vai trò
API endpoint	backend/api/*.ashx	Nhận request HTTP từ frontend.
Controller	backend/App_Code/Controllers	Xử lý nghiệp vụ và trả JSON.
Model	backend/App_Code/Models	Mô tả dữ liệu nghiệp vụ.
Core	backend/App_Code/Core	Kết nối database và hàm tiện ích.
View	frontend/index.html, assets	Giao diện người dùng.
SQL	backend/sql	Tạo database, dữ liệu mẫu và migration.

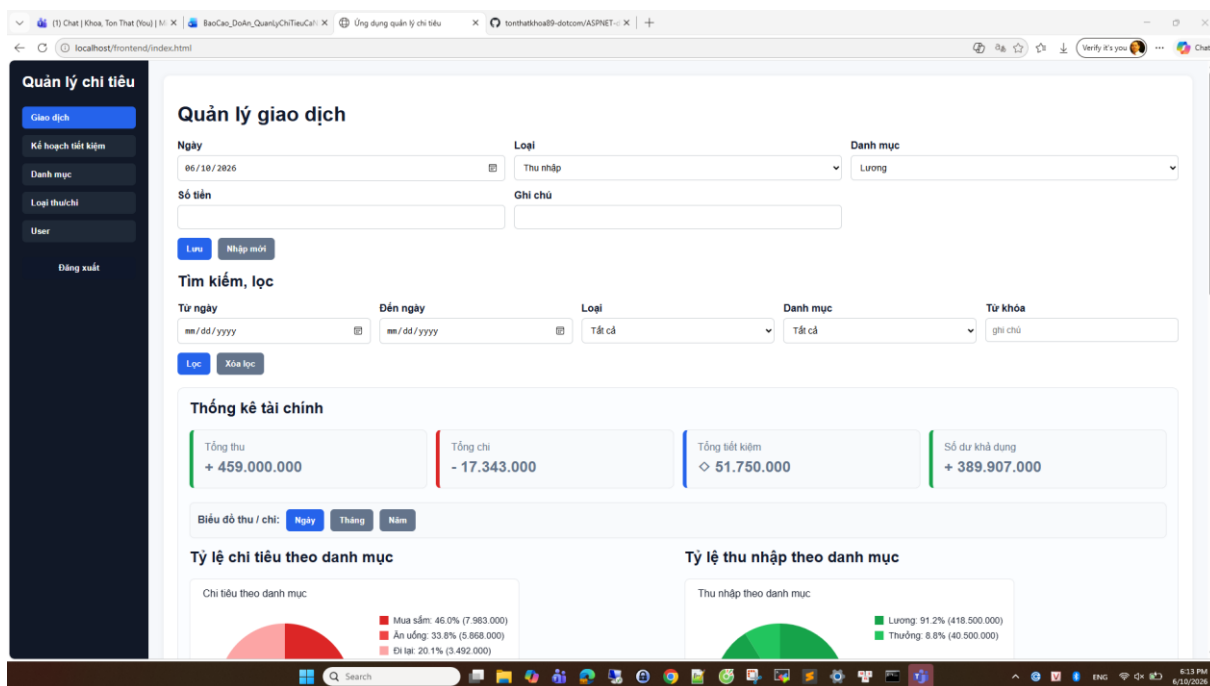
2.8. Thiết kế giao diện

Giao diện được thiết kế theo dạng dashboard một trang. Bên trái là sidebar chứa menu chức năng, bên phải là vùng nội dung hiển thị form, bộ lọc, bảng dữ liệu, thống kê và biểu đồ. Cách bố trí này giúp người dùng không phải chuyển qua nhiều trang, thao tác nhanh và dễ theo dõi trạng thái hiện tại.

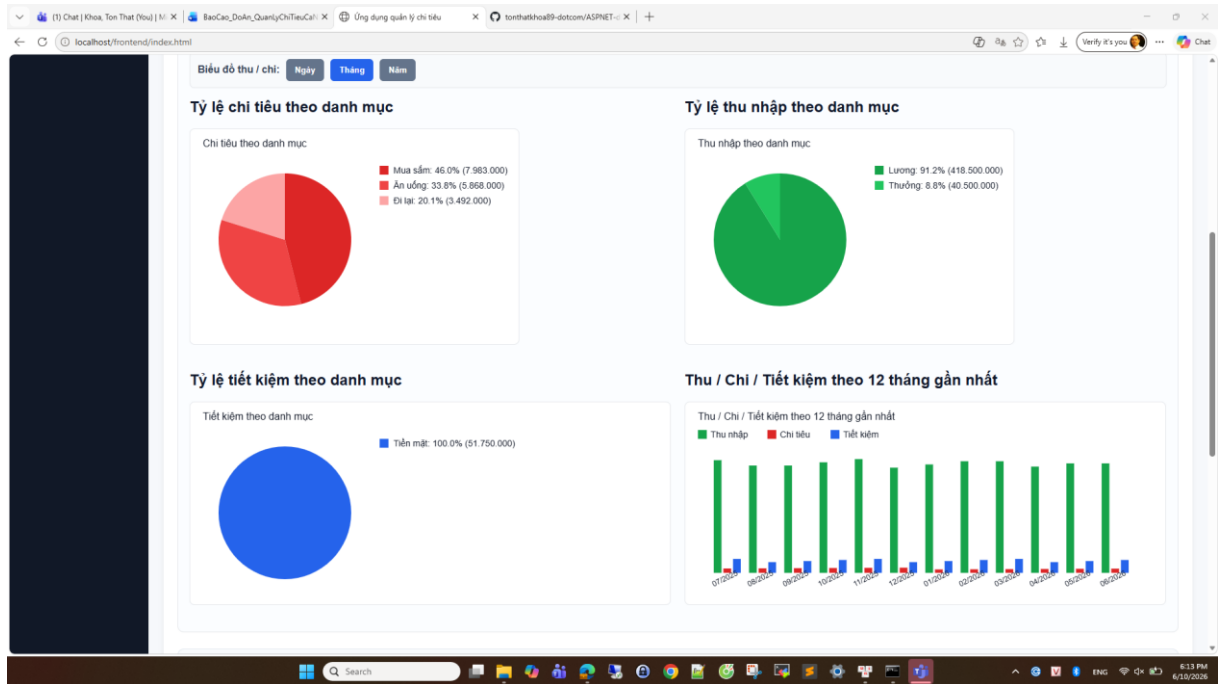
Màu sắc được phân biệt theo ý nghĩa nghiệp vụ: khoản thu hiển thị màu xanh lá, khoản chi màu đỏ, khoản tiết kiệm màu xanh dương. Các cảnh báo ngân sách sử dụng nền vàng hoặc đỏ tùy mức độ. Phân quyền Admin/User cũng được thể hiện bằng badge trong bảng quản lý user.



Hình 2.8.1: Màn hình đăng nhập



Hình 2.8.2: Màn hình Quản lý giao dịch



Hình 2.8.3: màn hình biểu đồ giao dịch

Hiển thị 10 giao dịch / trang Trang trước Trang 1/11 - Tổng 108 giao dịch Trang sau

Ngày	Loại	Danh mục	Số tiền	Ghi chú	Sửa	Xóa
2026-06-27	CHI	Mua sắm	- 452.000	Mua sắm ngày 26/06/2026	Sửa	Xóa
2026-06-19	Thu	Thưởng	+ 2.600.000	Thưởng tháng 06/2026	Sửa	Xóa
2026-06-15	CHI	Đi lại	- 198.000	Đi lại ngày 16/06/2026	Sửa	Xóa
2026-06-09	Tiết kiệm	Tiền mặt	> 3.000.000	Tiết kiệm tiền mặt tháng 06/2026	Sửa	Xóa
2026-06-04	Thu	Lương	+ 23.000.000	Lương tháng 06/2026	Sửa	Xóa
2026-06-03	CHI	Ăn uống	- 332.000	Ăn uống ngày 04/06/2026	Sửa	Xóa
2026-05-27	CHI	Mua sắm	- 435.000	Mua sắm ngày 28/05/2026	Sửa	Xóa
2026-05-19	Thu	Thưởng	+ 2.000.000	Thưởng tháng 05/2026	Sửa	Xóa
2026-05-15	CHI	Đi lại	- 190.000	Đi lại ngày 16/05/2026	Sửa	Xóa
2026-05-09	Tiết kiệm	Tiền mặt	> 2.750.000	Tiết kiệm tiền mặt tháng 05/2026	Sửa	Xóa

Đề tài: ứng dụng quản lý chi tiêu | GVHD: TS Đoàn Phước Miền | Lớp: DK24TTB0171 | Họ và tên: Tôn Thất Khoa | MSSV: 170124090

Hình 2.8.4: màn hình các giao dịch đã tạo

Quản lý chi tiêu

- Giao dịch
- Kế hoạch tiết kiệm**
- Danh mục
- Loại thu chi
- User
- Đăng xuất

Lập kế hoạch tiết kiệm

Đặt mục tiêu tiết kiệm, theo dõi tiến độ, cảnh báo khi vượt ngân sách và xem gợi ý tiết kiệm cơ bản theo dữ liệu tháng hiện tại.

Tên mục tiêu: Số tiền mục tiêu: Ngân sách chi tiêu/tháng:

Ngày bắt đầu: Ngày đặt mục tiêu: Hoạt động:

Thu tháng này
+ 25.500.000

Chi tháng này
- 982.000

Tiết kiệm tháng này
◇ 3.000.000

Số dư tháng này
+ 21.518.000

Cảnh báo ngân sách

Chưa có cảnh báo ngân sách trong tháng này.

Gợi ý tiết kiệm

Danh mục chi nhiều nhất là "Mua sắm" chiếm khoảng 46,0% tổng chi. Có thể đặt giới hạn riêng cho danh mục này.

Danh sách mục tiêu

Mục tiêu	Thời gian	Mục tiêu	Đã tiết kiệm	Còn thiếu	Tiến độ	Ngân sách/tháng	Trạng thái
Quỹ dự lịch admin	2025-01-01 → 2026-06-30	30 000 000	51 750 000	0	100.0%	12 000 000	Đang theo dõi Sửa Xóa
Quỹ đầu tư admin	2025-06-01 → 2026-12-31	50 000 000	37 500 000	12 500 000	75.0%	15 000 000	Đang theo dõi Sửa Xóa

Hình 2.8.5: Màn hình Lập kế hoạch tiết kiệm

Quản lý chi tiêu

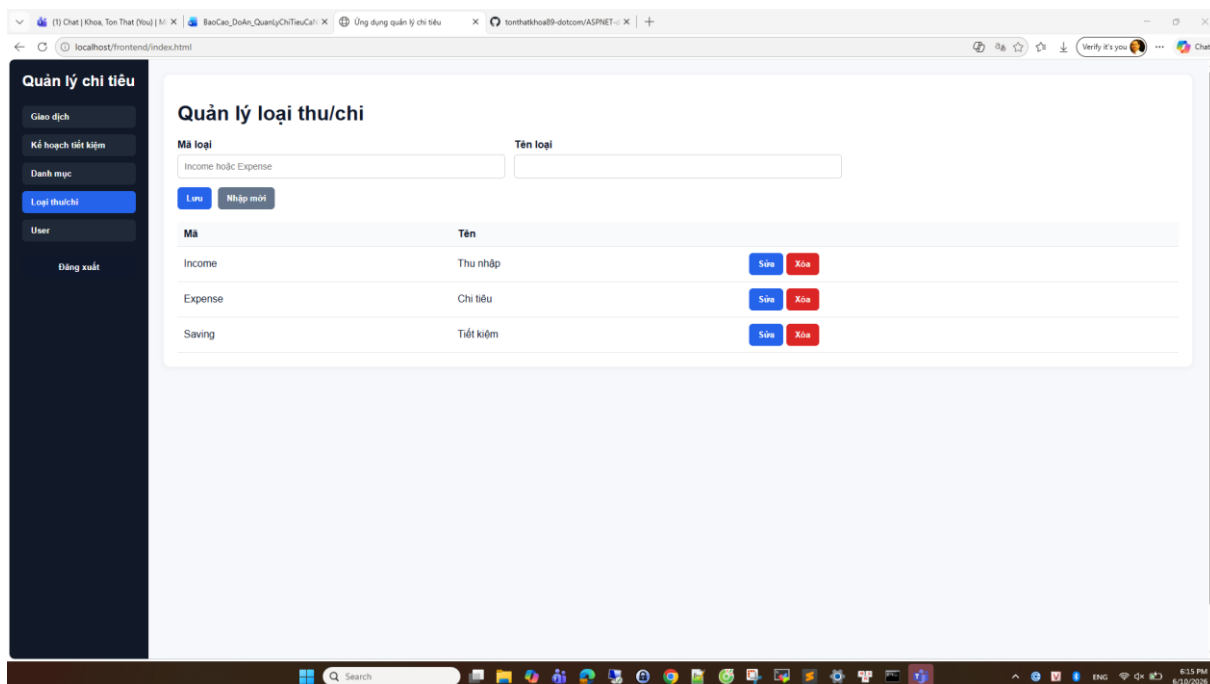
- Giao dịch
- Kế hoạch tiết kiệm
- Danh mục**
- Loại thu chi
- User
- Đăng xuất

Quản lý danh mục

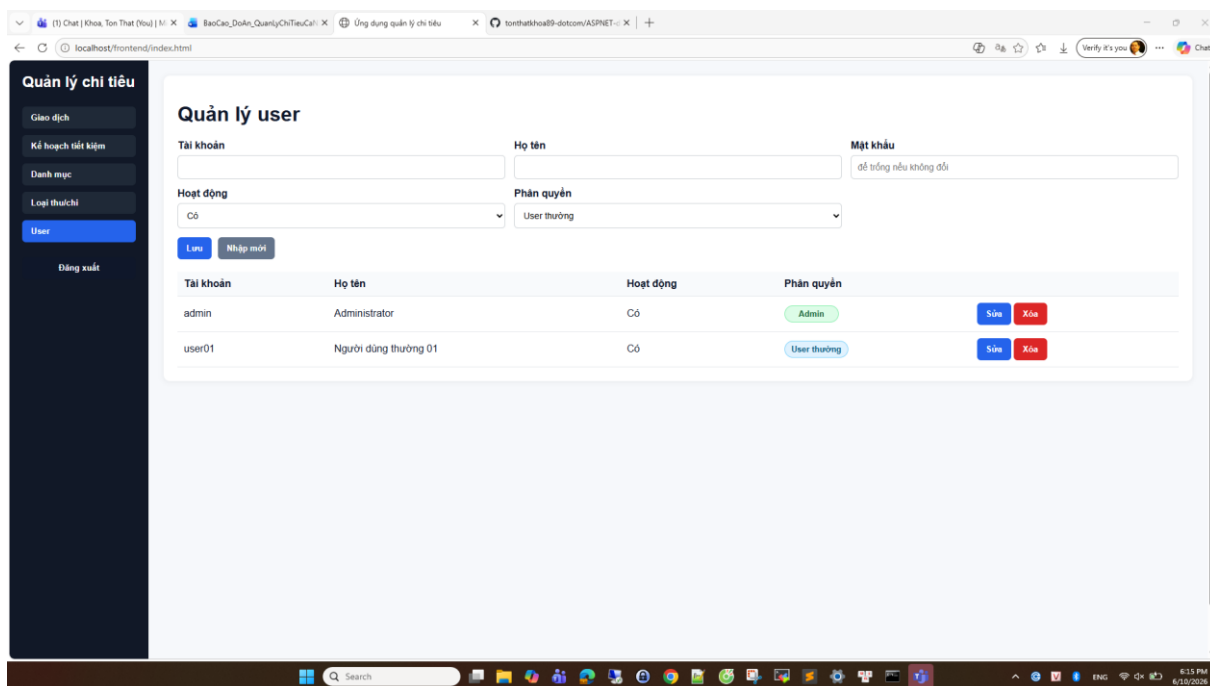
Loại: Tên danh mục: Hoạt động:

Loại	Danh mục	Hoạt động	
Thu nhập	Lương	Có	Sửa Xóa
Thu nhập	Thưởng	Có	Sửa Xóa
Chi tiêu	Ăn uống	Có	Sửa Xóa
Chi tiêu	Đi lại	Có	Sửa Xóa
Chi tiêu	Mua sắm	Có	Sửa Xóa
Tiết kiệm	Cổ phiếu	Có	Sửa Xóa
Tiết kiệm	Quỹ tiết kiệm	Có	Sửa Xóa
Tiết kiệm	Tiền mặt	Có	Sửa Xóa
Tiết kiệm	Vàng	Có	Sửa Xóa

Hình 2.8.6: Màn hình quản lý danh mục giao dịch



Hình 2.8.7 Màn hình quản lý loại thu / chi



Hình 2.8.8 Màn hình quản lý user

Bảng 2.8.1: Danh sách màn hình chính của hệ thống

STT	Màn hình	Người dùng	Mô tả
1	Đăng nhập	Admin/User	Nhập username và password để vào hệ thống.
2	Giao dịch	Admin/User	Thêm, sửa, xóa, lọc, phân trang và xem thống kê giao dịch.
3	Kế hoạch tiết kiệm	Admin/User	Tạo mục tiêu, xem tiến độ, cảnh báo ngân sách và gợi ý tiết kiệm.
4	Danh mục	Admin	Quản lý danh mục theo loại thu/chi/tiết kiệm.
5	Loại thu/chi	Admin	Quản lý loại giao dịch Income, Expense, Saving.
6	User	Admin	Quản lý tài khoản, trạng thái và quyền Admin/User.

2.9. Thiết kế API

Bảng 2.9.1: Danh sách API chính

API	Phương thức	Controller	Mô tả
login.ashx	POST	LoginController	Đăng nhập và tạo session.
logout.ashx	POST/GET	LogoutController	Hủy session đăng

			nhập.
me.ashx	GET	MeController	Kiểm tra user đang đăng nhập.
transactions.ashx	GET/POST	TransactionsController	Lọc, phân trang, thêm, sửa, xóa giao dịch.
statistics.ashx	GET	StatisticsController	Trả số liệu thống kê và biểu đồ.
savingsGoals.ashx	GET/POST	SavingsGoalsController	Quản lý mục tiêu tiết kiệm.
categories.ashx	GET/POST	CategoriesController	Đọc/quản trị danh mục.
transactionTypes.ashx	GET/POST	TransactionTypesController	Đọc/quản trị loại giao dịch.
users.ashx	GET/POST	UsersController	Quản lý user, chỉ Admin.

2.10. Luồng kiểm tra số dư lũy kế

Một yêu cầu quan trọng của hệ thống là không cho phép thêm mới hoặc chỉnh sửa khoản Chi tiêu và Tiết kiệm vượt số dư hiện có. Số dư hiện có được tính theo nguyên tắc lũy kế: số dư các tháng trước được cộng dồn sang tháng hiện tại. Khi kiểm tra một giao dịch thuộc tháng 06/2026, hệ thống tính tổng Income trừ Expense và Saving từ trước đến cuối tháng 06/2026.

Đối với giao dịch đang sửa, backend loại giao dịch hiện tại khỏi phép tính để tránh trường hợp giao dịch tự trừ chính nó. Nếu số tiền mới lớn hơn số dư lũy kế, API trả lỗi 400 và frontend hiển thị thông báo cho người dùng.



Hình 2.6: Luồng thêm/sửa giao dịch và kiểm tra số dư

2.11. Kiểm tra dữ liệu nhập

Hệ thống kiểm tra dữ liệu ở cả frontend và backend. Frontend kiểm tra nhanh các trường bắt buộc như ngày giao dịch, loại, danh mục, số tiền; kiểm tra ngày đến không được trước ngày bắt đầu trong bộ lọc; kiểm tra ngày đạt mục tiêu tiết kiệm phải sau ngày bắt đầu. Backend thực hiện lại các kiểm tra này để đảm bảo an toàn khi request được gửi trực tiếp.

Đối với kế hoạch tiết kiệm, TargetDate phải lớn hơn StartDate. Đối với giao dịch, Amount phải lớn hơn 0. Đối với khoản chi và khoản tiết kiệm, Amount không được vượt số dư lũy kế. Đối với user, Admin không được tự bỏ quyền admin của chính mình.

2.12 Thiết kế dữ liệu mẫu

Để phục vụ demo, hệ thống có file SQL tạo dữ liệu mẫu cho hai tài khoản: admin và user01. Tài khoản admin có quyền quản trị và dữ liệu giao dịch 6 tháng; tài khoản user01 là user thường và có dữ liệu giao dịch 1 năm. Cả hai tài khoản đều có kế hoạch tiết kiệm để có thể kiểm tra tiến độ, cảnh báo và gợi ý tiết kiệm.

Dữ liệu mẫu được giới hạn đến ngày 01/06/2026. Ghi chú giao dịch không dùng tiền tố “[Mẫu...]” để dữ liệu nhìn tự nhiên hơn trong giao diện. Danh mục tiết kiệm mẫu chỉ dùng “Tiền mặt” để tránh làm phức tạp yêu cầu quản lý nhiều loại tài sản. Các file SQL cũng có phần xóa dữ liệu cũ trong phạm vi mẫu để có thể chạy lại nhiều lần mà không bị trùng.

Bảng 2.12.1: Tài khoản và dữ liệu mẫu

Tài khoản	Mật khẩu	Quyền	Dữ liệu mẫu
admin	admin123	Admin	Giao dịch 6 tháng, kế hoạch Quỹ du lịch và Quỹ đầu tư.
user01	user01123	User thường	Giao dịch 1 năm, kế hoạch Quỹ dự phòng và Mua laptop.

2.13. Cách thức triển khai

Ứng dụng có thể triển khai thủ công hoặc bằng script tự động. Với cách thủ công, người dùng cài SQL Server Express, bật IIS và ASP.NET 4.8, copy source vào C:\inetpub\wwwroot\PersonalFinanceOffline, chỉnh connection string trong backend/Web.config, chạy các file SQL, tạo Application Pool và convert thư mục backend thành Application.

Với cách tự động, người dùng giải nén setup/scriptdeploy.zip, chạy run_deploy.bat, nhập tên instance SQL Server. Nếu không nhập, script dùng

SQLEXPRESS mặc định. Script sẽ tự copy source, chỉnh Web.config, chạy SQL, tạo App Pool, cấp quyền database và restart IIS.

CHƯƠNG 3: KẾT QUẢ NGHIÊN CỨU

3.1. Các chức năng đã hoàn thành

Sau quá trình phân tích, thiết kế và cài đặt, hệ thống đã hoàn thành các chức năng chính theo mục tiêu đề tài. Người dùng có thể đăng nhập, nhập giao dịch thu/chi/tiết kiệm, lọc dữ liệu, xem thống kê và lập kế hoạch tiết kiệm. Admin có thể quản lý danh mục, loại thu chi và tài khoản người dùng.

Hệ thống đã hỗ trợ phân quyền Admin/User. User thường chỉ nhìn thấy Giao dịch và Kế hoạch tiết kiệm, không thấy menu Danh mục, Loại thu/chi và User. Backend cũng chặn thao tác quản trị bằng RequireAdmin, do đó người dùng thường không thể tự gọi API để thay đổi dữ liệu cấu hình.

Bảng 3.1.1: Tổng hợp kết quả chức năng

Nhóm	Kết quả đạt được
Đăng nhập	Đăng nhập bằng tài khoản admin/user01, lưu session và trả quyền isAdmin.
Giao dịch	CRUD, lọc, tìm kiếm, phân trang 10/20/50, validate số dư.
Thống kê	Tổng thu, tổng chi, tiết kiệm, số dư và biểu đồ theo thời gian.
Kế hoạch tiết kiệm	CRUD mục tiêu, tiến độ, cảnh báo ngân sách và recommendation.
Phân quyền	Admin/User rõ ràng ở cả frontend và backend.
Triển khai	Có script deploy và cleanup trên IIS/SQL Server.

3.2. Kịch bản kiểm thử

Bảng 3.2.1: kịch bản kiểm thử chức năng

Mã	Kịch bản	Dữ liệu kiểm thử	Kết quả mong đợi
TC01	Đăng nhập admin	admin/admin123	Vào hệ thống và thấy đầy đủ menu.
TC02	Đăng nhập user thường	user01/user01123	Chỉ thấy Giao dịch và Kế hoạch tiết kiệm.
TC03	Thêm giao dịch thu	Loại Thu nhập, số tiền > 0	Lưu thành công, số dư tăng.
TC04	Thêm khoản chi vượt số dư	Expense lớn hơn số dư lũy kế	Bị chặn và hiển thị lỗi.
TC05	Thêm tiết kiệm vượt số dư	Saving lớn hơn số dư lũy kế	Bị chặn và hiển thị lỗi.
TC06	Lọc ngày sai	Đến ngày trước Từ ngày	Bị chặn và thông báo lỗi.
TC07	Kế hoạch ngày sai	TargetDate <= StartDate	Bị chặn và thông báo lỗi.
TC08	User gọi API quản trị	User thường gọi users.ashx	Backend trả lỗi không có quyền.

3.3. Đánh giá giao diện và trải nghiệm

Giao diện hệ thống được xây dựng đơn giản, phù hợp mục tiêu đồ án. Người dùng nhìn thấy các chức năng chính ở sidebar. Các biểu mẫu nhập liệu được đặt ở đầu màn hình, danh sách dữ liệu và biểu đồ nằm phía dưới. Màu sắc thu/chi/tiết kiệm được phân biệt rõ ràng giúp người dùng nhanh chóng nhận biết loại giao dịch.

Phần kế hoạch tiết kiệm có các thẻ tổng quan tháng, danh sách cảnh báo ngân sách và gợi ý tiết kiệm. Điều này giúp người dùng không chỉ ghi nhận giao dịch mà còn có thông tin hỗ trợ ra quyết định. Các thông báo lỗi được trả từ backend và hiển thị lại trên frontend, giúp người dùng hiểu lý do thao tác thất bại.

3.4. Đánh giá triển khai

Ứng dụng chạy được trên IIS với backend ASP.NET Framework 3.8 và SQL Server Express/SQL Server. Việc convert thư mục backend thành Application là bước quan trọng để IIS xử lý .ashx và App_Code. Cấu hình connection string trong backend/Web.config giúp backend kết nối đúng database.

Script auto deploy giúp giảm các thao tác thủ công. Script có thể yêu cầu nhập tên instance SQL Server, mặc định là SQLEXPRESS nếu người dùng để trống. Sau khi hoàn tất, người dùng truy cập <http://localhost/frontend/index.html> để sử dụng ứng dụng.

3.5. Hạn chế còn tồn tại

Do phạm vi đề án, hệ thống vẫn còn một số hạn chế. Ứng dụng chưa hỗ trợ nhiều ví tiền hoặc nhiều loại tài sản khác nhau. Chưa có chức năng import/export Excel. Chưa có cơ chế reset mật khẩu qua email. Giao diện biểu đồ được xây dựng bằng canvas cơ bản, chưa sử dụng thư viện biểu đồ chuyên sâu.

Việc bảo mật mật khẩu đã sử dụng SHA-256 nhưng chưa kết hợp salt riêng cho từng người dùng. Trong thực tế, nên sử dụng thuật toán băm mật khẩu chuyên dụng như PBKDF2, bcrypt hoặc Argon2. Ngoài ra, hệ thống mới chạy local/offline, chưa xử lý các vấn đề triển khai production như HTTPS, logging tập trung hoặc backup tự động.

CHƯƠNG 4: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

4.1. Kết luận

Đề tài “Ứng dụng quản lý thu chi cá nhân” đã hoàn thành mục tiêu xây dựng một website quản lý tài chính cá nhân chạy offline trên nền tảng ASP.NET Framework 4.8, IIS và SQL Server. Hệ thống đáp ứng các chức năng cốt lõi: đăng nhập, phân quyền, quản lý giao dịch, thống kê, biểu đồ, kế hoạch tiết kiệm, quản lý danh mục, loại thu chi và user.

Về mặt kỹ thuật, đề tài đã áp dụng mô hình MVC-style để tổ chức mã nguồn rõ ràng hơn: Model mô tả dữ liệu, Controller xử lý API và View là frontend HTML/CSS/JS. Ứng dụng sử dụng ADO.NET để truy cập SQL Server, Session để xác thực và phân quyền, JavaScript thuần để gọi API và render giao diện.

Về mặt nghiệp vụ, hệ thống hỗ trợ kiểm tra số dư lũy kế khi người dùng tạo khoản chi hoặc khoản tiết kiệm. Cơ chế này giúp dữ liệu hợp lý hơn và phản ánh đúng nguyên tắc số dư tháng trước được cộng dồn sang tháng sau. Ngoài ra, chức năng kế hoạch tiết kiệm giúp người dùng đặt mục tiêu, theo dõi tiến độ và nhận cảnh báo ngân sách.

4.2. Hướng phát triển

Trong tương lai, hệ thống có thể phát triển thêm chức năng quản lý nhiều ví tiền, nhiều tài khoản ngân hàng hoặc nhiều nguồn tiền khác nhau. Việc phân tách ví tiền sẽ giúp người dùng theo dõi tiền mặt, tài khoản ngân hàng, thẻ tín dụng và khoản đầu tư một cách chính xác hơn.

Hệ thống cũng có thể bổ sung chức năng import/export Excel, in báo cáo PDF, sao lưu và phục hồi dữ liệu. Đối với giao diện, có thể sử dụng thư viện biểu đồ local để tăng khả năng trực quan hóa dữ liệu, cho phép so sánh thu chi theo nhiều giai đoạn và xuất biểu đồ vào báo cáo.

Về bảo mật, hướng phát triển cần nâng cấp cơ chế lưu mật khẩu bằng thuật toán có salt và cấu hình HTTPS khi triển khai thực tế. Đồng thời có thể bổ sung audit log

để ghi nhận các thao tác thêm, sửa, xóa dữ liệu, giúp quản trị viên theo dõi lịch sử thay đổi trong hệ thống.

Về triển khai, có thể đóng gói ứng dụng thành bộ cài đặt hoàn chỉnh, tự động kiểm tra IIS, .NET Framework, SQL Server và quyền App Pool. Khi đó người dùng cuối chỉ cần chạy bộ cài đặt và nhập tên instance SQL Server để triển khai ứng dụng nhanh chóng.

4.3. Tài liệu tham khảo

- [1] Microsoft, “ASP.NET Web Handlers Overview”, Microsoft Learn.
- [2] Microsoft, “ADO.NET Overview”, Microsoft Learn.
- [3] Microsoft, “SQL Server Documentation”, Microsoft Learn.
- [4] Microsoft, “Internet Information Services (IIS) Documentation”, Microsoft Learn.

PHỤ LỤC A: CẤU TRÚC MÃ NGUỒN

Mã nguồn được lưu trữ trên github tại url: <https://github.com/tonthatkhoa89-dotcom/ASPNET-dk24tt80171-tonthatkhoa-quanlychitieu>

Mã nguồn được tổ chức để người sử dụng có thể nhanh chóng tìm thấy các thành phần chính. Thư mục frontend chứa giao diện người dùng, thư mục backend chứa API, Controller, Model, Core và script SQL. Thư mục setup chứa các script triển khai tự động, còn thesis chứa tài liệu báo cáo.

Bảng A: Mô tả các thư mục trên github

Thư mục/File	Mô tả
setup/scriptdeploy.zip	Chứa run_deploy.bat, run_cleanup.bat và PowerShell script tự động triển khai.
src/PersonalFinanceOffline/frontend/index.html	View chính của hệ thống, chứa layout đăng nhập và các màn hình quản lý.
src/PersonalFinanceOffline/frontend/assets/app.js	JavaScript gọi API, xử lý form, phân quyền giao diện, phân trang và biểu đồ.
src/PersonalFinanceOffline/frontend/assets/style.css	Định dạng giao diện offline, không phụ thuộc CDN.
src/PersonalFinanceOffline/backend/api	Các endpoint .ashx được frontend gọi.
src/PersonalFinanceOffline/backend/App_Code/Controllers	Controller xử lý nghiệp vụ theo MVC-style.
src/PersonalFinanceOffline/backend/App_Code/Models	Các lớp mô tả dữ liệu như UserModel, TransactionModel, SavingsGoalModel.
src/PersonalFinanceOffline/backend/App_Code/Core	Các tiện ích dùng chung như Db.cs và WebUtil.cs.

src/PersonalFinanceOffline/backend/sql	Script tạo database, dữ liệu mẫu và phân quyền.
thesis/doc	Chứa file báo cáo .docx.

Việc tổ chức thư mục rõ ràng giúp thuận tiện cho việc đưa source lên GitHub, chạy lại chương trình và bảo trì.

PHỤ LỤC B: PHÂN TÍCH LỖI THƯỜNG GẶP KHI TRIỂN KHAI

Bảng B: Lỗi triển khai thường gặp và cách xử lý

Lỗi	Nguyên nhân	Cách xử lý
Không load được .ashx	IIS chưa bật ASP.NET 4.8 hoặc backend chưa Convert to Application.	Bật ASP.NET 4.8, ISAPI Extensions, convert backend thành Application.
Login trả HTML thay vì JSON	API lỗi 500 nhưng frontend cố parse JSON.	Mở DevTools Network để xem response thật.
Cannot open database	App Pool chưa có quyền vào database.	Cấp quyền SQL cho IIS APPPOOL tương ứng.
CSS không load	Đường dẫn frontend/assets sai hoặc cache trình duyệt.	Kiểm tra URL CSS, Ctrl+F5.
Lỗi tiếng Việt	File C# hoặc SQL không lưu UTF-8.	Lưu UTF-8 with BOM, cấu hình globalization UTF-8.
SQL SSL certificate	ODBC Driver 18 yêu cầu trust certificate.	Thêm TrustServerCertificate=True trong connection string.

error		
404 sau deploy	Sai physical path hoặc copy lồng thư mục.	Kiểm tra path trong IIS.
User vẫn thấy menu admin	Frontend chưa nhận isAdmin mới hoặc cache JS.	Ctrl+F5 và kiểm tra me.ashx trả isAdmin.

Trong quá trình triển khai thử nghiệm, lỗi phổ biến nhất là backend chưa được convert thành Application. Khi đó IIS có thể không biên dịch App_Code đúng cách hoặc không xử lý được file `.ashx`. Đây là lý do bước convert backend to Application được đưa vào checklist deploy.

Lỗi quyền SQL cũng thường gặp khi chạy trên IIS. Mặc dù database tồn tại và có thể truy cập bằng SSMS, App Pool identity vẫn cần được cấp quyền riêng. Nếu không, backend sẽ báo lỗi login failed hoặc cannot open database.

Các lỗi frontend thường liên quan đến cache. Sau khi thay file CSS hoặc JavaScript, người dùng cần nhấn Ctrl+F5 để trình duyệt tải lại file mới. Trong quá trình phát triển, có thể thêm query version vào đường dẫn CSS/JS để tránh cache cũ.